



SEO Playbook

Everything that was implemented on the Cartify store, why each piece matters, and how to do it yourself on the next project.

Prepared for	Hamza — Cartify (Django marketplace)
Date	15 August 2026
Scope	Technical SEO, on-page SEO, structured data, social previews
Status	Implemented and verified on all page types

Contents

- 1 How search engines actually see your site
- 2 What was changed — the summary
- 3 The building blocks, explained one by one
 - 3.1 The title tag
 - 3.2 The meta description
 - 3.3 The H1 and heading order
 - 3.4 Canonical URLs — the duplicate content fix
 - 3.5 The robots meta tag (noindex)
 - 3.6 robots.txt
 - 3.7 sitemap.xml
 - 3.8 Open Graph and the og-image
 - 3.9 Structured data (JSON-LD)
 - 3.10 Image SEO
 - 3.11 Internal links and breadcrumbs
 - 3.12 URL structure
- 4 The Django pattern that makes it all reusable
- 5 Two bugs found along the way
- 6 How to check your own work
- 7 What to do next, in priority order
- 8 One-page cheat sheet

How to read this document

Sections marked **DONE** are already live in your codebase. Sections marked **YOUR TURN** are things only you can decide or supply. Every block includes a short "do it yourself" recipe you can copy onto any future site.

1 How search engines actually see your site

Almost every SEO rule makes sense once you understand that Google does three separate jobs, in this order. If you fail at step one, nothing you do at step three matters.

STAGE	WHAT HAPPENS	WHAT CAN GO WRONG
1. Crawl	A bot ("Googlebot") requests your URLs, like a browser with no JavaScript patience and no login.	It can't find your pages, or it wastes its time on your cart and admin pages.
2. Index	It reads the HTML and decides <i>what the page is about</i> and whether it deserves to be stored.	Every page has the same title, or the page is thin, or ten URLs show the same content.
3. Rank	When someone searches, it picks which stored pages to show and in what order.	Your content doesn't match the words people actually type, or a competitor's is better.

Everything in section 3 is really just helping Google at one of those three stages:

- **Crawl helpers:** sitemap.xml, robots.txt, internal links
- **Index helpers:** title, meta description, H1, canonical, noindex, structured data
- **Rank helpers:** keyword-matched copy, page speed, images, unique content

The single most useful habit

Open any page, press `Ctrl+U` (view source), and read the first 30 lines. That is roughly what Google sees. If you can't tell what the page is about from those lines, neither can Google.

2 What was changed — the summary

New files

FILE	PURPOSE
<code>products/sitemaps.py</code>	Generates <code>/sitemap.xml</code> automatically from the database — homepage, all 10 categories, and every active in-stock product.
<code>templates/robots.txt</code>	Instructions for crawlers: what to skip, where the sitemap lives.
<code>products/templatetags/seo_extras.py</code>	Three small helpers used by the templates: readable category names, safe meta-description trimming, and JSON escaping.
<code>static/og-image.png</code>	1200×630 branded image shown when a Cartify link is pasted into WhatsApp, Facebook or LinkedIn.

Edited files

FILE	CHANGE
<code>templates/base.html</code>	A complete SEO head section that every page inherits — description, canonical, robots, Open Graph, Twitter card, favicon, and site-wide structured data.
<code>templates/products/home.html</code>	Dynamic titles per category and per search, a real <code><h1></code> , canonical logic, breadcrumb and product-list structured data, lazy-loaded images.
<code>templates/products/productdetail.html</code>	Keyword-rich title, product-specific description and share image, full Product + Breadcrumb schema, a visible breadcrumb trail.
15 cart / account / seller templates	Unique titles plus <code>noindex</code> , <code>nofollow</code> so private pages stay out of search results.
<code>Cartify/urls.py</code> , <code>settings.py</code>	Routes for <code>/sitemap.xml</code> and <code>/robots.txt</code> ; registered Django's sitemap app.
<code>products/views.py</code>	Two bug fixes (see section 5).

Verified, not assumed

Every page type was rendered and inspected: all return HTTP 200, every page has exactly one `<h1>`, every title is unique and under 60 characters, every JSON-LD block parses as valid JSON, `/sitemap.xml` returns 12 URLs, and `/robots.txt` serves correctly.

3 The building blocks, explained one by one

Each block below follows the same shape: *what it is*, *why it matters*, *what was done on Cartify*, and *how you do it yourself next time*.

3.1 The title tag DONE

WHAT IT IS

The `<title>` in the page head. It becomes the blue clickable headline in Google results and the text in the browser tab.

WHY IT MATTERS

It is the strongest single on-page signal of what a page is about, and it is what convinces a human to click. Two rules: **keep it under about 60 characters** (Google cuts it off after that) and **make every page's title different**. Identical titles across pages tell Google your pages are interchangeable.

WHAT WAS DONE

Before, the homepage said "Cartify - Your All in one Store" no matter what you were looking at. Now the title changes with the page:

Home	→	Online Shopping in Pakistan Cartify	(37 chars)
Category	→	Buy Electronics Online in Pakistan Cartify	(44 chars)
Product	→	Playstation 5 - Buy Online in Pakistan Cartify	(48 chars)
Search	→	Search results for "play" Cartify	(35 chars)

DO IT YOURSELF

The formula that works for e-commerce is `Primary keyword + qualifier + | Brand`. Put the words people search first, your brand last. In Django:

```
<!-- base.html: the default -->
<title>{% block title %}Cartify | Online Shopping in Pakistan{% endblock %}</title>

<!-- any child template overrides it -->
{% block title %}{{ product.name }} - Buy Online in Pakistan | Cartify{% endblock %}
```

3.2 The meta description DONE

WHAT IT IS

```
<meta name="description" content="Shop electronics, fashion, home and kitchen...">
```

WHY IT MATTERS

It is **not** a ranking factor — Google has said so directly. It is the grey summary text under your title in the results, so it is a *click-through* factor. A good one can double your traffic from the same ranking position. Aim for **140–160 characters**; longer gets truncated.

WHAT WAS DONE

Every page type gets its own description. Product pages build theirs from the real product data:

```
{% block meta_description %}Buy {{ product.name }} online in Pakistan for  
PKR {{ product.price }}. {{ product.description|meta_text:100 }}{% endblock %}
```

The `meta_text` filter (in `seo_extras.py`) squashes line breaks and cuts the text at a word boundary so you never get a description that ends mid-word.

DO IT YOURSELF

Write it like ad copy, not like a summary. Include the main keyword, a reason to click (price, free returns, delivery), and never duplicate it across pages. If you leave it out entirely, Google invents one from your page text — usually badly.

3.3 The H1 and heading order DONE

WHAT IT IS

The `<h1>` is the visible main heading of the page. `<h2>` and `<h3>` are subheadings beneath it. This is the page's table of contents.

WHY IT MATTERS

Headings tell Google (and screen readers) the structure and the topic. The rule is **exactly one H1 per page**, describing that page, followed by H2s for sections. Do not pick a heading level because of how big it looks — that is what CSS is for.

WHAT WAS DONE

Several pages had *zero* H1s — the headings were styled `<div>`s, which carry no meaning at all. The homepage hero, cart heading, login heading and role heading are now real `<h1>` elements. Product cards on the listing became `<article>` blocks with an `<h2>` linked title. Pages with no natural visible heading (checkout, profile) got a screen-reader-only H1 using this class:

```
/* visible to Google and screen readers, invisible on screen */
.sr-only {
  position: absolute; width: 1px; height: 1px;
  overflow: hidden; clip: rect(0,0,0,0); white-space: nowrap;
}
```

DO IT YOURSELF

On any page ask: "what is the one thing this page is about?" That sentence is your H1. Then check with `Ctrl+F` in view-source that `<h1` appears exactly once.

3.4 Canonical URLs — the duplicate content fix DONE

WHAT IT IS

```
<link rel="canonical" href="https://yoursite.com/?category=electronics">
```

A tag that says: "if you reached this content by any other URL, *this* is the real address."

WHY IT MATTERS

This is the single most misunderstood tag in e-commerce SEO, and the most valuable. The same product list is reachable at many URLs:

```
/                /?category=all        /?page=1
/?category=all&page=1  /?search=            (and so on)
```

To Google those are five different pages with identical content. It then has to guess which one to rank, and it splits your credibility across all five. A canonical tag collapses them into one.

WHAT WAS DONE

The default in `base.html` is the page's own clean path with no query string:

```
{% block canonical %}{{ request.scheme }}://{{ request.get_host }}{{ request.path }}{% endblock %}
```

The homepage overrides it, because there the query string is meaningful — a category page is genuinely a different page, but `?category=all` is not:

URL VISITED	CANONICAL POINTS TO	WHY
<code>/?category=electronics</code>	<code>/?category=electronics</code>	Genuinely unique content — keep it
<code>/?category=all</code>	<code>/</code>	Same as the homepage — merge it
<code>/?page=2</code>	<code>/?page=2</code>	Page 2 has different products — keep it
<code>/?search=shoes</code>	<code>/</code>	Thin page, also marked noindex

DO IT YOURSELF

Rule of thumb: **if two URLs show the same thing, one of them must canonical to the other.** And always include a canonical even on unique pages ("self-referencing") — it protects you when someone links to your page with tracking parameters like `?fbclid=123`.

3.5 The robots meta tag (noindex) DONE

WHAT IT IS

```
<meta name="robots" content="noindex, nofollow">
```

WHY IT MATTERS

Not every page should be in Google. Your checkout page, someone's cart, the seller dashboard — these have no search value, and a store with 40 junk pages and 10 good ones looks like a low-quality site. Removing the junk raises the average.

The four values, decoded

index / noindex

May this page appear in search results?

follow / nofollow

Should Google follow the links on this page to discover other pages?

Use `noindex, follow` when the page itself is worthless but its links are useful (search results, an empty blog). Use `noindex, nofollow` for private pages behind a login.

WHAT WAS DONE

Fifteen templates were marked `noindex, nofollow`: cart, checkout, order confirmation, profile, wishlist, and all five seller pages. Search results are `noindex, follow`. The blog is `noindex, follow` until it has real posts — flip that once you publish.

DO IT YOURSELF

```
{% block robots %}noindex, nofollow{% endblock robots %}
```

Add that one line near the top of any private template. That is the whole job.

3.6 robots.txt DONE

WHAT IT IS

A plain text file at `yoursite.com/robots.txt` — the first thing every crawler requests. It controls **crawling**, not indexing.

The classic mistake

`robots.txt` and `noindex` are *not* the same thing, and using the wrong one backfires.

robots.txt Disallow = "don't visit this." The page can still appear in Google (from external links) — just with no description, because Google was never allowed to read it.

noindex = "visit it, read it, but don't list it."

If you block a page in robots.txt, Google can never see its noindex tag. **Never use both on the same page.** Use noindex for pages you want gone, robots.txt for whole areas you want crawlers to ignore.

WHAT WAS DONE

```
User-agent: *
Disallow: /admin/
Disallow: /cart/
Disallow: /seller/
Disallow: /users/profile/
Disallow: /*?search=

Sitemap: https://yoursite.com/sitemap.xml
```

It is served through Django rather than as a static file, so the sitemap line always matches whatever domain you deploy to:

```
path('robots.txt', TemplateView.as_view(
    template_name='robots.txt', content_type='text/plain'))
```

WHY "CRAWL BUDGET" MATTERS

Google gives every site a limited amount of crawling per day. If it spends that budget on 1,000 cart URLs, your new products sit undiscovered. Blocking the noise means the budget goes to pages that can earn money.

3.7 sitemap.xml DONE

WHAT IT IS

An XML file listing every URL you want indexed, optionally with the date each was last changed. It is a menu you hand to Google, so it doesn't have to find everything by following links.

```
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://cartify.pk/products/playstation-5/</loc>
    <lastmod>2026-08-15</lastmod>
    <changefreq>daily</changefreq>
    <priority>0.8</priority>
  </url>
</urlset>
```

WHY IT MATTERS

New products get found in hours instead of weeks. And in Google Search Console you can see "submitted 240 / indexed 180" — that gap is one of the most useful diagnostics you will ever get.

WHAT WAS DONE

The critical part: it is **generated from your database**, not written by hand. Add a product, it appears in the sitemap. Set it to closed or zero stock, it disappears. Nobody has to remember to update anything.

```
# products/sitemaps.py
class ProductSitemap(Sitemap):
    changefreq = "daily"
    priority = 0.8
    protocol = "https"

    def items(self):
        # only products a shopper can actually buy
        return Product.objects.filter(status="active", stock__gt=0)

    def lastmod(self, obj):
        return obj.created_at

    def location(self, obj):
        return reverse("productDetail", args=[obj.id])
```

Three sitemaps are registered — static pages, the 10 category pages, and products. Currently 12 URLs; it grows on its own.

DO IT YOURSELF

1. Add `'django.contrib.sitemaps'` to `INSTALLED_APPS`.
2. Write a `Sitemap` class with `items()` and `location()`.
3. Wire the URL: `path('sitemap.xml', sitemap, {'sitemaps': sitemaps})`.
4. Only ever list URLs that return 200 and are indexable — never list a noindex or redirected page.

3.8 Open Graph and the og-image DONE

WHAT IT IS

Open Graph is a set of `<meta property="og:...">` tags invented by Facebook and now read by WhatsApp, LinkedIn, Slack, Discord, iMessage and X. They control the preview card that appears when someone pastes your link.

WHY IT MATTERS

This is not technically SEO — it is *social CTR*. But for a store in Pakistan, WhatsApp sharing is likely a bigger traffic source than Google in year one. A link with no preview looks like spam; a link with a product photo, name and price gets clicked.

WHAT WAS DONE

```
<meta property="og:type"           content="product">
<meta property="og:title"         content="Playstation 5 - PKR 200000 | Cartify">
<meta property="og:description"   content="Take your gaming experience...">
<meta property="og:image"         content="https://.../media/ps5.jpeg">
<meta property="og:url"           content="https://.../products/.../">
<meta name="twitter:card"         content="summary_large_image">
```

Product pages use the actual product photo. Every other page falls back to `static/og-image.png` — a 1200×630 branded card that was generated for you showing the Cartify logo, "Online Shopping in Pakistan", the category list and three trust badges.

The og-image spec worth memorising

1200 × 630 pixels, PNG or JPG, under 1 MB, and an **absolute URL** (`https://...` , never `/static/...`). Keep important text away from the edges — different platforms crop differently. SVG does not work; most platforms ignore it.

DO IT YOURSELF

Test with Facebook's Sharing Debugger and LinkedIn's Post Inspector. Both cache aggressively — if you change the image you must click "Scrape Again" or the old one keeps showing for weeks.

3.9 Structured data / JSON-LD DONE

WHAT IT IS

A block of JSON in your page that describes the page's content in a vocabulary Google understands (schema.org). Your HTML says "PKR 200000" in a styled div; the JSON-LD says "this is a *Product*, its *price* is 200000, the *currency* is PKR, and it is *in stock*."

WHY IT MATTERS

This is what produces **rich results** — the price, stock status, star ratings and breadcrumb trail shown directly in the search listing. Rich results take up more space and get clicked far more than plain blue links. It is the highest-leverage markup on an e-commerce site.

WHAT WAS DONE

SCHEMA TYPE	WHERE	WHAT IT EARNS YOU
Organization	Every page	Brand name, logo, knowledge panel eligibility
WebSite + SearchAction	Every page	A search box inside your Google listing
Product + Offer	Product pages	Price, currency, in-stock badge in results
BreadcrumbList	Home, category, product	Home > Electronics > Playstation 5 instead of a raw URL
ItemList	Listing pages	Helps Google understand the page is a product grid

Here is the shape of the Product block, with the Django variables left in:

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "{{ product.name }}",
  "image": ["{{ product.image }}"],
  "sku": "{{ product.id }}",
  "offers": {
    "@type": "Offer",
    "price": "{{ product.price }}",
    "priceCurrency": "PKR",
    "availability": "{% if product.stock > 0 %}https://schema.org/InStock
                    {% else %}https://schema.org/OutOfStock{% endif %}"
  }
}
</script>
```

The rule that gets sites penalised — read this one twice

Structured data must describe what is actually visible on the page, and it must be true.

Your product page currently shows three hard-coded reviews ("Sarah Johnson", "Mike Chen") and a "4.7 out of 5" rating that is not connected to any database. **I deliberately left `AggregateRating` and `Review` out of the markup for that reason.** Marking up invented reviews breaches Google's structured data policy and can trigger a manual action — a human reviewer at Google demoting your entire domain. It is one of the most common ways small stores destroy their own rankings.

Either build a real review model, or delete the fake reviews. Once ratings are real, adding `AggregateRating` gets you gold stars in the search results, which is one of the biggest click-through wins available.

DO IT YOURSELF

Write the JSON, paste the rendered page into Google's **Rich Results Test**, fix what it flags, repeat. Note the `json_string` filter in `seo_extras.py` — a product name containing a quote mark would otherwise produce broken JSON and silently kill the whole block.

3.10 Image SEO DONE

WHAT WAS DONE, AND WHY EACH PART

```

```

alt

Describes the image for blind users and for Google Images — a real traffic source for products. Describe the picture; don't stuff keywords.

width and height

Lets the browser reserve the right space *before* the image loads, so the page doesn't jump around. That jumping is measured by Google as **Cumulative Layout Shift**, one of the three Core Web Vitals that affect ranking.

loading="lazy"

Images below the fold only download when the user scrolls near them. Applied from the 5th product onward — the first four load immediately with `fetchpriority="high"`, because they are what the visitor sees first and Google times that (**Largest Contentful Paint**).

decoding="async"

Stops image decoding from blocking the rest of the page rendering.

Never lazy-load your hero image

It is the most common self-inflicted speed problem. Lazy-loading the biggest visible image *delays* the exact thing Google is measuring. Lazy-load everything below the fold, eager-load what is above it.

3.11 Internal links and breadcrumbs DONE

WHY IT MATTERS

Google discovers pages by following links, and it judges a page's importance partly by how many internal links point at it. A product reachable only from page 7 of a listing looks unimportant.

WHAT WAS DONE

- A visible breadcrumb on product pages: *Home > Electronics > Playstation 5*, backed by matching `BreadcrumbList` markup so Google shows that trail in results.
- The footer now links to the main category pages from every single page on the site.
- Card titles are links, so the anchor text is the product name rather than "View Product". Anchor text tells Google what the destination is about — "click here" tells it nothing.

3.12 URL structure YOUR TURN

This one is not done, because it needs a database change and is your call.

```
Today:    /products/ed914ba2-3313-4432-8b16-0785e64bef6f/  
Better:   /products/playstation-5/
```

The UUID carries no meaning. A slug puts your keyword in the URL, and — just as important — makes the link readable when someone shares it. Good URLs are short, lowercase, hyphenated, and describe the content.

To do it: add a `SlugField` to the Product model, populate it from the name on save, switch the URL pattern to match on slug, and 301-redirect the old UUID URLs so you keep any links that already exist.

4 The Django pattern that makes it all reusable

Everything above works through one mechanism: **template inheritance with named blocks**. Understand this and you can add SEO to any Django site in an afternoon.

`base.html` defines a default for each SEO tag. Any child template can replace any single one without touching the others:

```
<!-- base.html - the defaults, used by every page -->
<title>{% block title %}Cartify | Online Shopping in Pakistan{% endblock %}</title>
<meta name="description" content="{% block meta_description %}Shop electronics...{% endblock %}">
<meta name="robots" content="{% block robots %}index, follow{% endblock %}">
<link rel="canonical" href="{% block canonical %}...{% request.path %}{% endblock %}">
{% block structured_data %}{% endblock %}
```

```
<!-- productdetail.html - override only what differs -->
{% extends 'base.html' %}

{% block title %}{{ product.name }} - Buy Online in Pakistan | Cartify{% endblock %}
{% block meta_description %}Buy {{ product.name }} for PKR {{ product.price }}...{% endblock %}
{% block og_image %}{{ product.image }}{% endblock %}
{% block structured_data %}
    <script type="application/ld+json">{ ..Product schema... }</script>
{% endblock %}
```

Why this matters for you

A new page inherits correct SEO automatically. You only think about SEO when a page is *special* — and making a page private is now literally one line: `{% block robots %}noindex, nofollow{% endblock %}`.

The blocks now available to you

BLOCK	OVERRIDE IT WHEN...
<code>title</code>	Always. Every page needs its own.
<code>meta_description</code>	Always, for public pages.
<code>robots</code>	The page is private, thin, or a search result.
<code>canonical</code>	Query parameters are meaningful (filters, pagination).
<code>og_title</code> , <code>og_description</code> , <code>og_image</code> , <code>og_type</code>	The page has its own image or needs a different share preview.
<code>twitter_title</code> , <code>twitter_description</code> , <code>twitter_image</code>	Same, for X/Twitter.
<code>structured_data</code>	The page describes a real-world thing: a product, an article, an event.
<code>extra_head</code>	Page-specific CSS or <code>rel=prev/next</code> links.

5 Two bugs found along the way

SEO work means reading every page as a machine would, which is why it so often surfaces bugs that browsing never reveals.

Bug 1 — the Sports & Outdoors category was broken in both directions

The view validated the category against the string `'sports-outdoor'`, while the database stores `'sports-outdoors'` — a missing "s".

- Clicking the capsule filtered on a value no product has → **an empty category page**.
- Visiting the correct `?category=sports-outdoors` failed the validation, so no filter was applied → **the full catalogue**, which is a duplicate of your homepage.

The fix removed the whole class of bug by validating against the model's own list instead of a hand-typed chain of ten comparisons:

```
# before – ten hand-typed strings, one of them wrong
if category == 'grocery-essentials' or category == 'toys-games' or ...:

# after – impossible to drift out of sync with the model
valid_categories = [slug for slug, _ in CATEGORY]
if category in valid_categories:
    products = products.filter(category=category)
```

Bug 2 — pagination had no ordering

Django was warning that paginating an unordered queryset gives inconsistent results. In SEO terms: the same product could appear on page 1 and page 3 of different crawls, while others were never shown at all. Fixed with `order_by('-created_at')` before paginating.

Still open — not an SEO issue, but it will cost you sales

`/cart/checkout/` raises `UnboundLocalError` when the cart is empty, because `delivery_fee` is assigned inside the loop over cart items. It works fine with items in the cart. Roughly a two-line fix in `cart/views.py`.

6 How to check your own work

The free tools that matter

TOOL	USE IT FOR
Google Search Console	The single most important one. Which queries you rank for, which pages are indexed and which were rejected and why. Free, and it is Google telling you directly.
Rich Results Test	Paste a URL or HTML; it validates your structured data and previews the rich result.
PageSpeed Insights	Core Web Vitals (LCP, CLS, INP) with specific fixes.
Facebook Sharing Debugger	Checks your Open Graph tags and refreshes the cached preview.
Bing Webmaster Tools	Same idea as Search Console; takes five minutes and Bing feeds ChatGPT search.
Screaming Frog (free to 500 URLs)	Crawls your whole site and lists every missing title, duplicate description or broken link.

The 60-second manual check on any page

1. Press `Ctrl+U` to view source.
2. `Ctrl+F` for `<title` — is it there, unique, under 60 characters?
3. `Ctrl+F` for `name="description"` — present and under 160?
4. `Ctrl+F` for `<h1` — exactly one?
5. `Ctrl+F` for `canonical` — does it point where you expect?
6. `Ctrl+F` for `noindex` — is it there when it should be, and *absent* when it shouldn't?

The mistake that costs the most

Shipping a site with `noindex` left on the live pages — normally copied over from a staging site. It removes you from Google entirely and can take weeks to recover. Check point 6 above on launch day, every time.

Verify the plumbing after deploying

```
yoursite.com/robots.txt    → should list your Disallow rules + Sitemap line
yoursite.com/sitemap.xml   → should list your real https:// URLs
```

Then submit the sitemap in Search Console under *Indexing* → *Sitemaps*.

Don't forget this after every CSS or image change

Your project runs with `DEBUG = False` and Django's manifest static storage, so you must run `python manage.py collectstatic` after changing anything in `static/` . If you skip it, pages return errors. This has already been run for you.

7 What to do next, in priority order

Ordered by impact per hour of effort. The first three are worth more than everything else combined.

#	ACTION	WHY IT IS AT THIS POSITION
1	Remove or replace the fake reviews	Risk elimination beats optimisation. Invented reviews can trigger a manual penalty on the whole domain. Real reviews then unlock star ratings, one of the biggest CTR wins there is.
2	Write real content — 300+ unique words per product, an intro paragraph per category	You cannot rank for words that do not exist on your site. One product and an empty blog is the actual ceiling right now; no amount of markup changes that.
3	Set up Search Console and submit the sitemap	Until you do, you are optimising blind. Twenty minutes of work.
4	Lock the canonical domain — set <code>ALLOWED_HOSTS</code> , force HTTPS, pick www or non-www and 301 the other	<code>ALLOWED_HOSTS = ["*"]</code> means any domain pointed at your server serves the whole site — textbook duplicate content. Also move <code>SECRET_KEY</code> into <code>.env</code> .
5	Add slugs to product URLs	Keywords in the URL, shareable links. Needs a migration and redirects.
6	Add trust pages — About, Contact, Shipping & Returns, Privacy, Terms	Google weighs these for E-E-A-T on stores handling money, and Merchant Center requires them.
7	Start the blog — buying guides, comparisons, "best X in Pakistan under Y"	Product pages only catch people ready to buy. Guides catch them while they are still deciding, which is far more traffic. Remember to remove the blog's <code>noindex</code> .
8	Google Merchant Center product feed	Free Shopping listings. Your Product schema already has the required fields.

Launch-day checklist

- ☐ Live pages do **not** contain `noindex` (check the homepage and a product page)
- ☐ `/robots.txt` loads and its Sitemap line shows your real domain
- ☐ `/sitemap.xml` loads and lists `https://` URLs on your real domain
- ☐ Sitemap submitted in Google Search Console and Bing Webmaster Tools
- ☐ A product URL passes Google's Rich Results Test with no errors
- ☐ A product link pasted into WhatsApp shows the image, title and price
- ☐ One domain only — www and non-www do not both serve the site
- ☐ `collectstatic` has been run and the site loads over HTTPS

8 One-page cheat sheet

Every public page needs these six things

<title>Keyword + qualifier | Brand</title>

<meta name="description" content="Ad copy with a reason to click">

<link rel="canonical" href="https://site.com/this-page/">

<meta name="robots" content="index, follow">

<h1>One per page, describes the page</h1>

<meta property="og:image" content="https://site.com/og.png">

≤ 60 chars, unique

≤ 160 chars, unique

absolute URL

or noindex if private

1200×630, absolute

Numbers worth memorising

ELEMENT	TARGET	NOTE
Title tag	50–60 chars	Truncated beyond that
Meta description	140–160 chars	Not a ranking factor; a click factor
H1 per page	Exactly 1	More than one dilutes the signal
og:image	1200 × 630 px	Absolute URL, under 1 MB
Product description	300+ words	Unique — never the manufacturer's copy
LCP (load speed)	under 2.5 s	Core Web Vital, affects ranking
Sitemap size	50,000 URLs max	Split into an index file beyond that

Decision table: which tag do I need?

I WANT TO...	USE
Keep a page out of Google	<code><meta name="robots" content="noindex"></code>
Stop crawlers wasting time on a whole folder	<code>Disallow:</code> in robots.txt
Merge duplicate URLs into one	<code><link rel="canonical"></code>
Help Google find new pages fast	<code>sitemap.xml</code> + internal links
Show price and stock in the search result	Product JSON-LD
Control the WhatsApp / Facebook preview	Open Graph tags
Stop the page jumping while images load	<code>width</code> and <code>height</code> on <code></code>

If you remember only one thing

Technical SEO gets you *eligible* to rank — it clears the obstacles. It cannot make a thin site rank. Cartify's technical foundation is now solid and largely automatic; from here the wins come from **content**: more products, better descriptions, and articles that answer what your customers are searching for before they are ready to buy.